

请记住

- 有许多理由需要写个自定的 new 和 delete，包括改善效能、对 heap 运用错误进行调试、收集 heap 使用信息。

条款 51：编写 new 和 delete 时需固守常规

Adhere to convention when writing new and delete.

条款 50 已解释什么时候你会想要写个自己的 operator new 和 operator delete，但并没有解释当你那么做时必须遵守什么规则。这些规则不难奉行，但其中一些并不直观，所以知道它们究竟是些什么很重要。

让我们从 operator new 开始。实现一致性 operator new 必得返回正确的值，内存不足时必得调用 new-handling 函数（见条款 49），必须有对付零内存需求的准备，还需避免不慎掩盖正常形式的 new ——虽然这比较偏近 class 的接口要求而非实现要求。正常形式的 new 描述于条款 52。

operator new 的返回值十分单纯。如果它有能力供应客户申请的内存，就返回一个指针指向那块内存。如果没有那个能力，就遵循条款 49 描述的规则，并抛出一个 bad_alloc 异常。

然而其实也不是非常单纯，因为 operator new 实际上不只一次尝试分配内存，并在每次失败后调用 new-handling 函数。这里假设 new-handling 函数也许能够做某些动作将某些内存释放出来。只有当指向 new-handling 函数的指针是 null，operator new 才会抛出异常。

奇怪的是 C++ 规定，即使客户要求 0 bytes，operator new 也得返回一个合法指针。这种看似诡异的行为其实是为了简化语言其他部分。下面是个 non-member operator new 伪码（pseudocode）：

```
void* operator new(std::size_t size) throw(std::bad_alloc)
{
    //你的 operator new 可能接受额外参数.
    using namespace std;
    if (size == 0) {
        //处理 0-byte 申请.
        size = 1;
        //将它视为 1-byte 申请.
    }
    while (true) {
        尝试分配 size bytes;
```

```
    if (分配成功)
        return (一个指针, 指向分配得来的内存);

    //分配失败; 找出目前的 new-handling 函数 (见下)
    new_handler globalHandler = set_new_handler(0);
    set_new_handler(globalHandler);

    if (globalHandler) (*globalHandler)();
    else throw std::bad_alloc();
}
}
```

这里的伎俩是把 0 bytes 申请量视为 1 byte 申请量。看起来有点黏搭搭地令人厌恶, 但做法简单、合法、可行, 而且毕竟客户多久才会发出一个 0 bytes 申请呢?

你也可能带着怀疑的眼光斜睨这份伪码 (pseudocode), 因为其中将 new-handling 函数指针设为 null 而后又立刻恢复原样。那是因为我们很不幸地没有任何办法可以直接取得 new-handling 函数指针, 所以必须调用 set_new_handler 找出它来。拙劣, 但有效——至少对单线程程序而言。若在多线程环境中你或许需要某种机锁 (lock) 以便安全处置 new-handling 函数背后的 (global) 数据结构。

条款 49 谈到 operator new 内含一个无穷循环, 而上述伪码明白表明出这个循环; "while (true)" 就是那个无穷循环。退出此循环的唯一办法是: 内存被成功分配或 new-handling 函数做了一件描述于条款 49 的事情: 让更多内存可用、安装另一个 new-handler、卸除 new-handler、抛出 bad_alloc 异常 (或其派生物), 或是承认失败而直接 return。现在, 对于 new-handler 为什么必须做出其中某些事你应该很清楚了。如果不那么做, operator new 内的 while 循环永远不会结束。

许多人没有意识到 operator new 成员函数会被 derived classes 继承。这会导致某些有趣的复杂度。注意上述 operator new 伪码中, 函数尝试分配 size bytes (除非 size 是 0)。那非常合理, 因为 size 是函数接受的实参。然而就像条款 50 所言, 写出定制型内存管理器的一个最常见理由是为针对某特定 class 的对象分配行为提供最优化, 却不是为了该 class 的任何 derived classes。也就是说, 针对 class X 而设计的 operator new, 其行为很典型地只为大小刚好为 sizeof(X) 的对象而设计。然而一旦被继承下去, 有可能 base class 的 operator new 被调用用以分配 derived class 对象:

```
class Base {
public:
    static void* operator new(std::size_t size) throw(std::bad_alloc);
    ...
};

class Derived: public Base //假设 Derived 未声明 operator new
{ ... };

Derived* p = new Derived; //这里调用的是 Base::operator new
```

如果 Base class 专属的 operator new 并非被设计用来对付上述情况（实际上往往如此），处理此情势的最佳做法是将“内存申请量错误”的调用行为改采标准 operator new，像这样：

```
void* Base::operator new(std::size_t size) throw(std::bad_alloc)
{
    if (size != sizeof(Base)) //如果大小错误，
        return ::operator new(size); //令标准的 operator new 起而处理。
    ... //否则在这里处理。
}
```

“等一下！”我听到你大叫，“你忘了检验 size 等于 0 这种病态但是可能出现的情况！”。是的，我没检验，但请你收回你的但是。测试依然存在，只不过它和上述的“size 与 sizeof(Base) 的检测”融合一起了。是的，C++ 在某种秘境中运行，而其中一个秘境就是它裁定所有非附属（独立式）对象必须有非零大小（见条款 39）。因此 sizeof(Base) 无论如何不能为零，所以如果 size 是 0，这份申请会被转交到 ::operator new 手上，后者有责任以某种合理方式对待这份申请。

译注：这里所谓非附属/独立式（freestanding）对象，指的是不以“某对象之 base class 成分”存在的对象。此处所言的这个规定，可参考《*Inside the C++ Object*》by Stanly Lippman, Addison Wesley, 1996。

如果你打算控制 class 专属之“arrays 内存分配行为”，那么你需要实现 operator new 的 array 兄弟版：operator new[]。这个函数通常被称为“array new”，因为很难想出如何发音“operator new[]”。如果你决定写个 operator new[]，记住，唯一需要做的一件事就是分配一块未加工内存（raw memory），因为你无法对 array 之内迄今尚未存在的元素对象做任何事情。实际上你甚至无法计算这个 array 将含多少个元素对象。首先你不知道每个对象多大，毕竟 base class 的 operator new[] 有可能经由继承被调用，将内存分配给“元素为 derived class 对象”的 array 使用，而你当然知道，derived class 对象通常比其 base class 对象大。

因此, 你不能在 `Base::operator new[]` 内假设 `array` 的每个元素对象的大小是 `sizeof(Base)`, 这也就意味你不能假设 `array` 的元素对象个数是 `(bytes 申请数)/sizeof(Base)`。此外, 传递给 `operator new[]` 的 `size_t` 参数, 其值有可能比“将被填以对象”的内存数量更多, 因为条款 16 说过, 动态分配的 `arrays` 可能包含额外空间用来存放元素个数。

这就是撰写 `operator new` 时你需要奉行的规矩。`operator delete` 情况更简单, 你需要记住的唯一事情就是 C++ 保证“删除 `null` 指针永远安全”, 所以你必须兑现这项保证。下面是 `non-member operator delete` 的伪码 (pseudocode):

```
void operator delete(void * rawMemory) throw()
{
    if (rawMemory == 0) return;    //如果将被删除的是个 null 指针,
                                   //那就什么都不做。
    现在, 归还 rawMemory 所指的内存;
}
```

这个函数的 `member` 版本也很简单, 只需要多加一个动作检查删除数量。万一你的 `class` 专属的 `operator new` 将大小有误的分配行为转交 `::operator new` 执行, 你也必须将大小有误的删除行为转交 `::operator delete` 执行:

```
class Base {          //一如以往, 但此刻重点在 operator delete
public:
    static void* operator new(std::size_t size) throw(std::bad_alloc);
    static void operator delete(void* rawMemory, std::size_t size) throw();
    ...
};

void Base::operator delete(void* rawMemory, std::size_t size) throw()
{
    if (rawMemory == 0) return;          //检查 null 指针。
    if (size != sizeof(Base)) {          //如果大小错误, 令标准版
        ::operator delete(rawMemory);    //operator delete 处理此一申请。
        return;
    }
    现在, 归还 rawMemory 所指的内存;
    return;
}
```

有趣的是, 如果即将被删除的对象派生自某个 `base class` 而后者欠缺 `virtual` 析构函数, 那么 C++ 传给 `operator delete` 的 `size_t` 数值可能不正确。这是“让你的 `base classes` 拥有 `virtual` 析构函数”的一个够好的理由; 条款 7 还提过一个更好

的理由。我就不岔开话题了，此刻只要你提高警觉，如果你的 base classes 遗漏 virtual 析构函数，operator delete 可能无法正确运作。

请记住

- operator new 应该内含一个无穷循环，并在其中尝试分配内存，如果它无法满足内存需求，就该调用 new-handler。它也应该有能力处理 0 bytes 申请。Class 专属版本则还应该处理“比正确大小更大的（错误）申请”。
- operator delete 应该在收到 null 指针时不做任何事。Class 专属版本则还应该处理“比正确大小更大的（错误）申请”。

条款 52: 写了 placement new 也要写 placement delete

Write placement delete if you write placement new.

placement new 和 placement delete 并非 C++ 兽栏中最常见的动物，如果你不熟悉它们，不要感到挫折或忧虑。回忆条款 16 和 17，当你写一个 new 表达式像这样：

```
Widget* pw = new Widget;
```

共有两个函数被调用：一个是用以分配内存的 operator new，一个是 Widget 的 default 构造函数。

假设其中第一个函数调用成功，第二个函数却抛出异常。既然那样，步骤一的内存分配所得必须取消并恢复旧观，否则会造成内存泄漏（memory leak）。在这个时候，客户没有能力归还内存，因为如果 Widget 构造函数抛出异常，pw 尚未被赋值，客户手上也就没有指针指向该被归还的内存。取消步骤一并恢复旧观的责任因此落到 C++ 运行期系统身上。

运行期系统会高高兴兴地调用步骤一所调用的 operator new 的相应 operator delete 版本，前提当然是它必须知道哪一个（因为可能有许多个）operator delete 该被调用。如果目前面对的是拥有正常签名式（signature）的 new 和 delete，这并不是问题，因为正常的 operator new：

```
void* operator new(std::size_t) throw(std::bad_alloc);
```

对应于正常的 operator delete：

```
void operator delete(void* rawMemory) throw();  
                                     //global 作用域中的正常签名式.  
void operator delete(void* rawMemory, std::size_t size) throw();  
                                     //class 作用域中典型的签名式.
```

因此, 当你只使用正常形式的 new 和 delete, 运行期系统毫无问题可以找出那个“知道如何取消 new 所作所为并恢复旧观”的 delete。然而当你开始声明非正常形式的 operator new, 也就是带有附加参数的 operator new, “究竟哪一个 delete 伴随这个 new”的问题便浮现了。

举个例子, 假设你写了一个 class 专属的 operator new, 要求接受一个 ostream, 用来志记 (logged) 相关分配信息, 同时又写了一个正常形式的 class 专属 operator delete:

```
class Widget {  
public:  
    ...  
    static void* operator new(std::size_t size, std::ostream& logStream)  
        throw(std::bad_alloc);           //非正常形式的 new  
    static void operator delete(void* pMemory std::size_t size)  
        throw();                         //正常的 class 专属 delete  
    ...  
};
```

这个设计有问题, 但在探讨原因之前, 我们需要先绕道, 扼要讨论若干术语。

如果 operator new 接受的参数除了一定会有那个 size_t 之外还有其他, 这便是个所谓的 *placement* new。因此, 上述的 operator new 是个 *placement* 版本。众多 *placement* new 版本中特别有用的一个是“接受一个指针指向对象该被构造之处”, 那样的 operator new 长相如下:

```
void* operator new(std::size_t, void* pMemory) throw();  
                                     //placement new
```

这个版本的 new 已被纳入 C++ 标准程序库, 你只要 #include <new> 就可以取用它。这个 new 的用途之一是负责在 vector 的未使用空间上创建对象。它同时也是最早的 *placement* new 版本。实际上它正是这个函数的命名根据: 一个特定位置上的 new。以上说明意味术语 *placement* new 有多重定义。当人们谈到 *placement* new, 大多数时候他们谈的是此一特定版本, 也就是“唯一额外实参是个 void*”, 少数时候才是指接受任意额外实参之 operator new。上下文语境往往也能够使意义不明晰的含糊话语清晰起来, 但了解这一点相当重要: 一般性术语 “*placement*